Group members: Ashwin Varkey 23115420

Chukwudi Williams 23061170

Manik Malik 23169717

Assignment 2: Core Performance, SIMD, Caches

1. Calculate the maximum achievable in-core performance Pmax of the AVX-vectorized double-precision STREAM triad on a Cascade Lake SP core at 2.3 GHz. The STREAM triad kernel is a[i] = b[i] + s * c[i]. What fraction of the peak DP performance can be achieved (theoretically)? Also calculate Pmax for the AVX-512 and scalar variants.

STREAM TRIAD kernel computes a vector operation $A(i) = B(i) + s * C(i)$

This operation involves two loads (array B and C), one store (array A) and one MULT and one ADD instruction per kernel execution.

AVX

LD	LD	ST	MULT FMA	ADD MULT FMA	This core has a max. of 5 instructions per cycle can be executed
LD B	LD C	ST A	MULT	ADD	
	FLOPS		2 (1 MULT and 1 ADD)		
	1 AVX iteration		1 cycle	4 loop iterations	
	4 loop iterations		8 FLOPS		
	Pmax		8 FLOPS/cycle		

Pmax for AVX =8*2.3 GHz= 18.4 GFlops/s

Peak DP performance (theoretically) =

Cascade Lake core uses AVX-512 so it has the ability to 8 DP operands

Peak DP performance =2*2*8=32 FLOPS/cycle= 73.6 GFlops/s

We calculated before that AVX variant has maximum achievable in-core performance Pmax=8 FLOPs/cycle. Fraction of DP performance= 8/32=0.25 (If the peak DP performance was for the AVX variant of Cascade Lake core then it is **0.5**)

AVX-512

AVX512 LD	AVX512 LD	AVX512 ST	AVX512 MULT AVX512 FMA	AVX512 ADD AVX512 MULT AVX512 FMA
LD B	LD C	ST A	MULT	ADD
	FLOPS		2 (1 MULT and 1 ADD)	
	1 AVX iteration		1 cycle	8 loop iterations
	8 loop iterations		16 FLOPS	
	Pmax		16 FLOPS/cycle	

Pmax for AVX-512 =16*2.3 GHz= 36.8 GFlops/s

Scalar

LD	LD	ST	ADD MULT FMA	ADD MULT FMA
LD B	LD C	ST A	MULT	ADD
	FLOPS		2 (1 MULT and 1 ADD)	
			1 cycle	1 loop iteration
	1 loop iteration		2 FLOPS	
	Pmax		2 FLOPS/cycle	

Pmax for Scalar =2*2.3 GHz= 4.6 GFlops/s

2(a) Assuming that the compiler applies no unrolling and no SIMD vectorization, calculate the maximum possible performance of the loop in Flops/cycle if the data comes from the L1 cache.

LOAD	ADD	MULT
Load A	ADD	MULT A and B
Load B		

Maximum possible performance of the loop in Flops/cycle= 2/3

2(b) Latency = LD = (4+1) + 5 CY (MULT) + 3 CY (ADD) = 13 CY

	Using AVX		
	LOAD	MULT	ADD
1	Load A1		
2	Load B1		
3	Load A2		
4	Load B2		
5	Load A3		
6	Load B3	MULT A1 and B1	
7	Load A4		
8	Load B4	MULT A2 and B2	
9	Load A5		
10	Load B5	MULT A3 and B3	
11	Load A6		ADD
12	Load B6	MULT A4 and B4	
13	Load A7		
14	Load B7	MULT A5 and B5	ADD
15	Load A8		
16	Load B8	MULT A6 and B6	
17			ADD
18		MULT A7 and B7	
19			
20		MULT A8 and B8	ADD
21			
22			
23			ADD
24			
25			
26			ADD
27			
28			
29			ADD
30			
31			
32			ADD
33			
34			

Peak performance= 16/34 Flops/cycle=0.47 Flops/cycle

2. c) For 1 AVX iteration we need 2 LOAD, 1 MUL, 1 ADD. We have 8 additional cycles for load but as we want steady state maximum throughput.

In part (a) we get 2/3 maximum flops/cycle. With AVX we will have additional setup time but the throughput will remain the same as 3 cycles/iteration instead the iteration will now perform 4x floating point operations.

4* (2/3) = 8/3 F/cy

Now for (b) part with N= 8 iterations our time to first result or latency has increased due to 8 additional cycles given in the question to load the results. So Latency now = 13+8 = 21. But the iterations will be N/4 due to AVX SIMD vectorization.

Cycles for 1 iteration = 3

Cycles for N-1 iterations = 3(N-1)

After adding the new latency = 21

Total cycles = 21 + 3(N-1)

But in case of AVX we have N/4 iterations so total cycles = 21 + 3(N/4 -1)

In our case N = 8, so total iteration / cycle = (8 /4) / 21+ 3 *(2-1) = 2/21+3 = 1/12 iteration/cycle

1 iteration we are doing 8 Floating point calculations = 8/12 = 2/3 so the performance == the Pmax of No unrolling and No SIMD and better then b part.

8-Wide SIMD

- Part (a) For 8-Wide SIMD units Maximum throughput = 8* 2/3 = 16/3 twice that with 4 wide SIMD.
- Part (b) for 8-Wide SIMD units Maximum throughput = 2* 2/3 = 4/3 twice that with 4 wide SIMD.

2 d) Now assume that the compiler performs a 2-way modulo unrolling of the loop, on top of AVX. Calculate the maximum expected performance with AVX if the data comes from the L1 cache. Would 4-way unrolling change anything? Why (not)?

If the compiler performs AVX vectorization plus 2-way modulo unrolling

S1 = s1 + a[i] * b[i];

S2 = s2 + a [i+1] * b [i+1];

The ADD latency issue will be somewhat reduced to 3 cycles per 2 AVX iterations

Every AVX iteration has two loads for A and B which means for 2 iterations->4 Loads Latency (4 cycles)

Bottleneck- LOAD

So 4*2 FLOPS for 2 AVX iterations->16 FLOPS/4 cycles= **4 FLOPS/Cycle (for AVX+2 way roll)**

S1 = s1 + a[i] * b[i];

S2 = s2 + a [i+1] * b [i+1];

S3 = s1 + a [i+2] * b [i+2];

S4 = s2 + a [i+3] * b [i+3];

S=S1+S2+S3+S4

If we use 4-way unrolling, the load pipeline continues to increase and remains the bottleneck, the maximum performance is still same.

2. e) in question 2 part a, At steady state we had 2 load 1 add and 1 Multi per iteration = 4 instruction per iteration. So 4 N total instructions without loop mechanism considered. With loop mechanism 3 instructions per loop is added so 4N + 3N = 7 N instructions. (N being the number of iterations required)

1 AVX iteration is also 2 Load, 1 Add and 1 Multi = 4

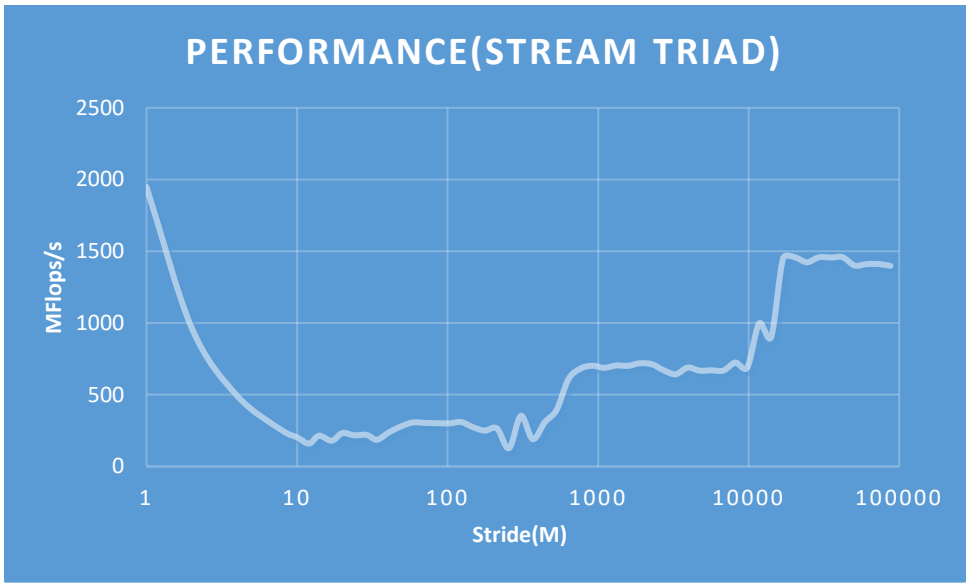
After SIMD and 2 way unrolling

Total iterations are N/4.

Hence, 4* N/4 (for execution of total iterations) + 3 *2 N/4(for loop mechanism) + 4(component for horizontal add of s after exiting final loop)

= N+ 3N/2 + 4 == 5N/2 (for large N, 4 can be ignored)

3)



Code:

```
#include <stdio.h>
#include <stdlib.h>
#include "timing.h"
#include <math.h>
```

```
int main(int argc, char** argv) {

    double wct_start, wct_end;
    int id, nt, k, i, j, M;
    int NITER, N;
    double t = 0;
    const double s = 1.00000000001;

    printf("\n\nVector Stride:  a[i] = b[i] + s * d[i]; \n");
    printf("M\t\t MFlop/s\n\n");

    N = 4e6;
    double* a = (double*)malloc(N * sizeof(double));
    double* b = (double*)malloc(N * sizeof(double));
    double* d = (double*)malloc(N * sizeof(double));

    int M_strides[55] = {
1,2,4,8,10,12,14,17,20,24,29,34,41,50,59,71,86,103,123,148,177,213,256,307,368,442,5
30,636,763,916,1099,1319,1583,1899,2279,2735,3281,3938,4725,5670,6804,8165,9798,1175
8,14110,16932,20318,24382,29258,35110,42132,50558,60670,72804,87364 };
    for (k = 0; k < N; ++k) {
        a[k] = 0.;
        b[k] = 0.;
        d[k] = 0.;
    }

    NITER = 1;
    for (j = 0; j < 55; j++) {
        M = M_strides[j];
        do {
            // time measurement
            wct_start = getTimeStamp();

            // repeat measurement often enough
            for (k = 0; k < NITER; ++k) {
                // This is the benchmark loop
                for (i = 0; i < N; i += M) {
                    // (b) vector a[i] = b[i] + s * d[i];
                    a[i] = b[i] + s * d[i];
                }
                // end of benchmark loop
                if (a[N / 2] < 0.) printf("%lf", a[N / 2]);
            }
            wct_end = getTimeStamp();
            NITER = NITER * 2;
        } while (wct_end - wct_start < 0.1); // at least 100 ms
        NITER = NITER / 2;
        double time = (wct_end - wct_start);
        double MFlop = (2 * N * (NITER / (M)))/(time*1.0e6);
        double MFlopPerSec = MFlop / (1.0e-3 * NITER);
        printf("%d\t %f\n", M, MFlop);
    }
    free(a);
    free(b);
    free(d);

    return 0;
}
```

The graph of performance versus stride values will showcase a decreasing trend as M increases, reflecting the decline in memory bandwidth and sudden spike in performance in certain areas before degrading again.

The performance of the loop with growing stride (M) will generally decrease. The change in performance is mainly attributed to memory access patterns and the cache hierarchy.

When M is 1 (no stride), the loop accesses memory locations consecutively, taking advantage of spatial locality and cache prefetching. This results in high cache utilization and memory bandwidth.

As M increases, the loop starts skipping elements, resulting in larger strides between memory accesses. Only each Mth element is being accessed. So performance goes down by a factor 1/M. This introduces irregular memory access patterns and reduces spatial locality. The cache prefetching mechanism becomes less effective, leading to increased cache misses and lower memory bandwidth.

Between 530 and 696 M- Stride value, the performance spikes because cache lines fits in L3 cache

Consequently, as M continues to grow from here, the performance decreases due to the diminishing cache effectiveness and encounter cache conflicts. The irregular memory access pattern causes more cache misses, resulting in longer memory access times. This behavior is expected because the stride causes the loop to access memory locations that are mapped to the same cache set, resulting in cache conflicts and degraded performance.

Towards the end we see an increase in the graph and stabilizing because the L2 cache is being accessed for cache lines (whole set of cache lines accessed fits in L2 cache).